FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Geofrey Miguel Erlano & Lance Kirsten Buenaventura
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.

7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:
   - Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
   - Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:
   - MS Word, Google Docs, or LibreOffice
   - Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:
   - Knowledge Representation & Reasoning
   - Case-Based Reasoning cycle
   - Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
   - Google Drive for shared documentation
   - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.

Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking

✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ “New case saved” or similar confirmation

✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud

- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System
A CBR system identifies the learning struggles of students. Student submits a math solution with repeated errors. System retrieves past students with similar error patterns (e.g., misusing formulas). It adapts feedback and personalized lessons based on successful interventions. **Learning:** System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

```
import json
import os
import copy
import math
import re
import shutil

# ==========================================
# 1. DATABASE MANAGEMENT
# ==========================================
DB_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "disaster_memory.json")

def starter_cases():
    """Return the three original classroom example cases."""
    starter_base = [
        {
            "id": "Local Memory - Super Typhoon Odette (2021)",
              "historical_context": "Typhoon Odette was a Category 5 super typhoon that caused widespread devastation in the Visayas and Mindanao. The main lesson learned was that communication lines went down immediately, leaving communities isolated. Pre-emptive evacuation saved thousands of lives.",
             "problem": {"disaster_type": "Typhoon", "intensity": 5.0, "location": "Visayas / Mindanao", "proximity_hazard": "Coastline"},
            "solution": {
                        "evacuation_strategy": "Pre-emptive Mandatory Evacuation 48hrs before landfall. Activate emergency sirens and deploy local police to enforce clearing of coastal/low-lying zones.",
                "shelter_capacity": 15000,
                   "shelter_description": "Reinforced concrete facilities (e.g., universities, indoor stadiums). Maintain backup generators.",
                "resource_allocation": {
                                "Survival": ["100,000 Liters of Drinking Water", "50,000 Meals Ready-to-Eat (MREs)"],
                            "Medical": ["5 Mobile Field Hospitals", "200 First-Aid Trauma Kits", "Cholera Vaccines"],
                             "Logistics": ["30 Rubber Rescue Boats", "10 High-clearance Military Trucks", "Clearing chainsaws"]
                }
```

```
            }
        },
        {
            "id": "Local Memory - Bohol Earthquake (2013)",
                 "historical_context": "A devastating Magnitude 7.2 earthquake struck Bohol,
destroying centuries-old churches and triggering landslides that isolated mountain towns.
Rescue efforts proved that open-field tent cities were necessary because people were terrified
of aftershocks collapsing indoor shelters.",
                  "problem": {"disaster_type": "Earthquake", "intensity": 7.2, "location":
"Visayas", "proximity_hazard": "Mountain"},
            "solution": {
                  "evacuation_strategy": "Immediate evacuation to open fields. Strictly cordon
off heritage structures and masonry houses. Deploy air assets for isolated mountain
barangays.",
                "shelter_capacity": 20000,
                  "shelter_description": "Modular tent cities in open areas. Ensure deep-well
water access.",
                "resource_allocation": {
                          "Survival": ["Air-dropped MREs and Bottled Water", "10,000 Thermal
Blankets"],
                    "Medical": ["Military Field Surgeons", "Crush-syndrome specialized trauma
units", "Psychological support teams"],
                   "Logistics": ["Military Helicopters (Airlift)", "Heavy Payloaders to clear
landslides", "Portable cell tower units (COWs)"]
                }
            }
        },
        {
            "id": "Local Memory - Metro Manila Monsoon Floods (2012)",
              "historical_context": "Weeks of relentless monsoon rains submerged Metro Manila,
paralyzing the capital. Many residents were trapped on their roofs for days. The response
highlighted the critical need for early water-level warnings and massive deployments of rubber
boats.",
                "problem": {"disaster_type": "Flood", "intensity": 3.0, "location": "Luzon",
"proximity_hazard": "River"},
            "solution": {
                    "evacuation_strategy": "Targeted evacuation of ground-floor residents in
low-lying zones. Activate municipal water pumping stations.",
                "shelter_capacity": 5000,
                 "shelter_description": "Elevated public schools and basketball courts. Ground
floors must be kept clear.",
                "resource_allocation": {
                        "Survival": ["50,000 Hot Meals via community kitchens", "Portable Water
Filtration systems"],
                    "Medical": ["Leptospirosis Prophylaxis (Doxycycline)", "Tetanus shots",
"First aid for lacerations"],
                          "Logistics": ["50 Rubber Rescue Boats (Outboard motors)", "Sandbag
deployment trucks", "Water utility repair teams"]
                }
            }
        }
    ]
    return starter_base


def normalize_solution(solution):
    """Convert legacy records without dropping resources or changing the source."""
    result = copy.deepcopy(solution)
    capacity = result.get("shelter_capacity")
    if isinstance(capacity, str):
        text = capacity.strip()
        if re.fullmatch(r"\d+|\d{1,3}(?:,\d{3})+", text):
```

```
            capacity = int(text.replace(",", ""))
        else:
                match = re.search(r"(\d[\d,]*)\s*(?:beds|displaced (?:individuals|residents))",
text)
            if not match:
                raise ValueError("A saved shelter capacity could not be read: " + text)
            capacity = int(match.group(1).replace(",", ""))
            result.setdefault("shelter_capacity_legacy", text)
                    result.setdefault("shelter_description",  text[match.end():].split("  ->",
1)[0].strip(" ."))
    if (isinstance(capacity, bool) or not isinstance(capacity, (int, float))
            or not math.isfinite(capacity) or capacity <= 0 or int(capacity) != capacity):
        raise ValueError("Shelter capacity must be a positive whole number.")
    result["shelter_capacity"] = int(capacity)
    result.setdefault("shelter_description", "Facility details were not recorded.")

    resources = result.get("resource_allocation", {})
    if isinstance(resources, list):
        grouped = {"Survival": [], "Medical": [], "Logistics": []}
        for item in resources:
            if not isinstance(item, str):
                raise ValueError("Resource entries must be text.")
            name = item.lower()
            if any(word in name for word in ("food", "water", "meal", "blanket")):
                category = "Survival"
            elif any(word in name for word in ("medical", "medicine", "first aid", "trauma")):
                category = "Medical"
            elif any(word in name for word in ("boat", "truck", "vehicle", "sandbag")):
                category = "Logistics"
            else:
                category = "Other"
            grouped.setdefault(category, []).append(item)
        resources = grouped
    if not isinstance(resources, dict):
        raise ValueError("Resource allocation must be a list or categorized dictionary.")
    for category in ("Survival", "Medical", "Logistics"):
        resources.setdefault(category, [])
    for category, items in resources.items():
        if not isinstance(items, list) or any(not isinstance(item, str) for item in items):
            raise ValueError("Each resource category must contain a list of text entries.")
        resources[category] = list(dict.fromkeys(items))
    result["resource_allocation"] = resources
    return result


def load_database():
    """Migrate older records and add a starter for any missing disaster type."""
    exists = os.path.exists(DB_FILE)
    if exists:
        with open(DB_FILE, encoding="utf-8") as file:
            original = json.load(file)
        if not isinstance(original, list):
            raise ValueError("The case base must be a JSON list.")
    else:
        original = []
    cases = copy.deepcopy(original)
    for case in cases:
        case["solution"] = normalize_solution(case["solution"])
        case["problem"].setdefault("proximity_hazard", "None")
    present_types = {case["problem"]["disaster_type"].strip().casefold() for case in cases}
    for starter in starter_cases():
        if starter["problem"]["disaster_type"].casefold() not in present_types:
```

```
            cases.append(starter)
    if not exists or cases != original:
        # Keep the user's original records recoverable before the first migration.
        backup = DB_FILE + ".legacy.bak"
        if exists and not os.path.exists(backup):
            shutil.copy2(DB_FILE, backup)
        save_database(cases)
    return cases


def save_database(case_base):
    """Replace the database only after the complete JSON has been written."""
    temporary = DB_FILE + ".tmp"
    with open(temporary, 'w', encoding="utf-8") as file:
        json.dump(case_base, file, indent=4, ensure_ascii=False, allow_nan=False)
    os.replace(temporary, DB_FILE)

# ========================================
# 2. INTENSITY & UI HELPERS
# ========================================
def display_scale_guide(disaster_type):
    print(f"\n--- 📋 {disaster_type.upper()} GUIDE ---")
    if disaster_type == "Earthquake":
        print(" 4.0 - 5.9 : Light to Moderate (Slight damage to buildings)")
        print(" 6.0 - 6.9 : Strong (Destructive in populated areas)")
        print(" 7.0 - 7.9 : Major (Serious damage over large areas)")
        print(" 8.0+      : Catastrophic (Severe widespread destruction)")
    elif disaster_type == "Typhoon":
        print(" 1.0 - 2.0 : Signal 1/2 (Minor/Moderate Threat)")
        print(" 3.0       : Signal 3 (Strong, Winds 121-170 km/h)")
        print(" 4.0       : Signal 4 (Severe, Winds 171-220 km/h)")
        print(" 5.0       : Signal 5 (Super Typhoon, Winds > 220 km/h)")
    elif disaster_type == "Flood":
        print(" 1.0 - 2.0 : Minor/Moderate (Knee-deep, impassable to vehicles)")
        print(" 3.0       : High (Waist-deep, ground-floor evacuation needed)")
        print(" 4.0       : Severe (Roof-level, immediate water rescue needed)")
        print(" 5.0       : Catastrophic (Widespread inundation)")

def get_intensity_category(disaster_type, intensity):
    if disaster_type == "Earthquake":
        if intensity < 6.0: return "Light to Moderate"
        elif intensity < 7.0: return "Strong"
        elif intensity < 8.0: return "Major"
        else: return "Catastrophic"
    elif disaster_type in ["Typhoon", "Flood"]:
        if intensity < 3.0: return "Minor to Moderate Threat"
        elif intensity < 4.0: return "High Danger"
        elif intensity < 5.0: return "Severe Threat"
        else: return "Catastrophic Level"
    return "Unknown"

# ========================================
# 3. INTERNET SEARCH FUNCTION (USGS API)
# ========================================
def search_internet_for_cases(intensity):
     print(f"\n[SYSTEM] 🌐 Connecting to the internet... Searching for Magnitude {intensity}
Earthquakes...")
    url = "https://earthquake.usgs.gov/fdsnws/event/1/query"
    params = {"format": "geojson", "minmagnitude": str(intensity - 0.5), "limit": 10}
    internet_cases = []

    try:
```

```
        # Online lookup is optional; the local CBR demonstration needs no packages.
        import requests
        response = requests.get(url, params=params, timeout=5)
        response.raise_for_status()
        for feature in response.json()["features"]:
            props = feature["properties"]
            mag = float(props["mag"])

            case = {
                "id": f"INTERNET RECORD: {props['title']}",
                       "historical_context": f"USGS reports a Magnitude {mag} earthquake at
{props['place']}. The response below is a classroom template, not an observed response from
USGS.",
                    "problem": {"disaster_type": "Earthquake", "intensity": mag, "location":
str(props["place"]), "proximity_hazard": "None"},
                "solution": {
                    "evacuation_strategy": "Immediate open-space evacuation. Do NOT use indoor
shelters due to aftershock collapse risks. Establish secure perimeters around damaged
high-rises.",
                    "shelter_capacity": max(1, int(mag * 2500)),
                    "shelter_description": "Tent cities in open parks (classroom estimate).",
                    "resource_allocation": {
                         "Survival": [f"{int(mag * 15000)}L Clean Water", "Portable Sanitation
Units"],
                           "Medical": ["Deploy Urban Search & Rescue (USAR)", "Ambulance fleet
for crush injuries"],
                       "Logistics": ["Heavy Excavators for debris", "Structural Engineers"]
                    }
                }
            }
            internet_cases.append(case)
        print(f"[SYSTEM] ✔ Found {len(internet_cases)} real-world records online!")
    except Exception:
        print("[SYSTEM] ⚠ Internet search failed. Relying only on local memory.")

    return internet_cases

# ==========================================
# 4. CBR LOGIC: RETRIEVE & ADAPT
# ==========================================
def retrieve_similar_case(new_problem, full_case_base):
    best_case = None
    highest_score = 0

    for case in full_case_base:
        score = 0
        past_prob = case["problem"]
        # Intensities from different disaster scales must never be compared.
                           if   past_prob.get("disaster_type",   "").strip().casefold()   !=
new_problem["disaster_type"].strip().casefold():
            continue

        score += 5
            if new_problem["location"].strip() and new_problem["location"].strip().lower() in
past_prob.get("location", "").lower():
            score += 3
               if new_problem["proximity_hazard"].strip().lower() not in ("", "none") and
new_problem["proximity_hazard"].strip().lower()      in      past_prob.get("proximity_hazard",
"").lower():
            score += 2

        intensity_diff = abs(past_prob.get("intensity", 0) - new_problem["intensity"])
```

```
        score += max(0, 5 - intensity_diff)

        if score > highest_score:
            highest_score = score
            best_case = case

    return best_case, highest_score

def adapt_solution(retrieved_solution, new_problem, past_problem):
    new_solution = normalize_solution(retrieved_solution)
    # A retained case's previous review does not count as review of this scenario.
    new_solution.pop("capacity_review", None)
    hazard = new_problem.get("proximity_hazard", "").lower()


    # --- ADD FRIENDLY TIPS BASE ---
    friendly_tips = [
            "🎒 Pack a 'Go-Bag' with your ID, important documents, medications, and enough
food/water for 3 days.",
          "📻 Keep your phone fully charged and have a battery-powered radio ready for local
announcements.",
          "🤝 Check on your neighbors, especially the elderly or those with disabilities, to
ensure they have a plan."
    ]

    # 1. Numeric capacity adaptation: the 50% increase is a classroom rule.
    original_capacity = new_solution["shelter_capacity"]
    increase_applied = new_problem["intensity"] > past_problem.get("intensity", 0) + 1.0
    if increase_applied:
                    new_solution["evacuation_strategy"]  =  "🚨  CRITICAL  UPGRADE:  "  +
str(new_solution["evacuation_strategy"])
        new_solution["shelter_capacity"] = math.ceil(original_capacity * 1.5)
    new_solution["capacity_adaptation"] = {
        "original_capacity": original_capacity,
        "recommended_capacity": new_solution["shelter_capacity"],
        "increase_percent": 50 if increase_applied else 0,
         "rule": "Increase by 50% when intensity exceeds the past case by more than 1.0; round
up.",
    }

    # 2. Add specific tips and protocols based on Disaster Type
    if new_problem["disaster_type"] == "Flood":
          friendly_tips.append("🔌 Unplug all electrical appliances and move valuables to the
second floor or higher ground.")
           friendly_tips.append("🦠 Avoid wading through floodwaters! They can carry diseases
like Leptospirosis or hide dangerous debris.")
    elif new_problem["disaster_type"] == "Earthquake":
         friendly_tips.append("🪑 Practice 'Drop, Cover, and Hold On' away from glass windows
and heavy furniture.")
          friendly_tips.append("🚪 Keep your shoes near your bed in case you need to evacuate
over broken glass at night.")
    elif new_problem["disaster_type"] == "Typhoon":
          friendly_tips.append("🪟 Secure loose outdoor items (like potted plants or chairs)
that could become flying debris in high winds.")
           friendly_tips.append("🕯️ Avoid using candles for light if the power goes out to
prevent fire hazards; stick to flashlights.")

    # 3. Hazard Proximity Protocols
    if "coast" in hazard or "sea" in hazard or "ocean" in hazard or "beach" in hazard:
            new_solution["evacuation_strategy"]  +=  "\n    🌊 [COASTAL PROTOCOL]: Enforce a
5-kilometer no-entry zone from the shoreline."
```

```
                new_solution["resource_allocation"]["Logistics"].extend(["Coast Guard Marine
Vessels"])
            friendly_tips.append("🌊 Since you are near the coast, find an evacuation center that
is inland and at least 3 stories high to avoid storm surges or tsunamis.")

        elif "river" in hazard or "dam" in hazard or "lake" in hazard:
            new_solution["evacuation_strategy"] += "\n   🌊 [RIVERINE PROTOCOL]: Monitor upstream
water levels. Evacuate communities within 1km of riverbanks."
              new_solution["resource_allocation"]["Logistics"].extend(["Motorized Rubber Boats",
"Sandbags"])
            friendly_tips.append("🌊 Avoid crossing river bridges. Flash floods can wash bridges
away in seconds without warning.")

        elif "mountain" in hazard or "slope" in hazard or "hill" in hazard:
              new_solution["evacuation_strategy"] += "\n     ⛰️ [MOUNTAIN PROTOCOL]: Activate
landslide early warning systems."
                new_solution["resource_allocation"]["Logistics"].extend(["Heavy Payloaders and
Earth-movers (Clearing)"])
            friendly_tips.append("⛰️ Be alert for unusual sounds like trees cracking or boulders
knocking together, which might indicate an impending landslide.")

        elif "fault" in hazard or "volcano" in hazard:
              new_solution["evacuation_strategy"] += "\n     🌋 [SEISMIC PROTOCOL]: Establish a
permanent danger zone. Shut down local gas lines immediately."
            friendly_tips.append("🌋 Know the exact location of your nearest open-space evacuation
area (like a park or empty field) far away from tall buildings.")

        new_solution["friendly_tips"] = friendly_tips
        for category, items in new_solution["resource_allocation"].items():
            new_solution["resource_allocation"][category] = list(dict.fromkeys(items))
        return new_solution


def review_solution(solution):
    """Let the user accept or revise capacity before the retain stage."""
    reviewed = copy.deepcopy(solution)
    recommended = reviewed["shelter_capacity"]
    print("\n--- 4. REVIEW AND REVISE SHELTER CAPACITY ---")
    print(f"Recommended capacity: {recommended:,} beds")
    while True:
        answer = input("Press Enter to accept, or enter a revised whole number of beds (e.g.,
8000): ").strip()
        if not answer:
            final_capacity = recommended
            break
        if re.fullmatch(r"[0-9]+", answer) and len(answer) <= 12 and int(answer) > 0:
            final_capacity = int(answer)
            break
        print("Please enter a positive whole number, or press Enter to accept.")
    reviewed["shelter_capacity"] = final_capacity
    reviewed["capacity_review"] = {
        "recommended_capacity": recommended,
        "final_capacity": final_capacity,
        "status": "revised" if final_capacity != recommended else "accepted",
    }
    print(f"[✔] Reviewed final shelter capacity: {final_capacity:,} beds")
    return reviewed

# ========================================
# 5. MAIN INTERACTIVE PROGRAM
# ========================================
def main():
```

```
    print("============================================================")
    print(" 🌟 HELPFUL DISASTER RESPONSE & COMMUNITY GUIDE 🌟")
    print("============================================================")

    print("Classroom CBR demonstration; capacity changes use a project-defined rule.")
    try:
        local_database = load_database()
    except (OSError, ValueError, KeyError, TypeError) as error:
        print(f"Could not load the case base: {error}")
        return
     print(f"Hello! I have loaded {len(local_database)} past disaster records to help keep your
community safe.")

    while True:
        # --- PHASE 1: NEW PROBLEM DESCRIPTION ---
        print("\n--- 1. TELL ME ABOUT THE SITUATION ---")
           d_type = input("What kind of disaster are we preparing for? (Earthquake, Typhoon,
Flood): ").strip().capitalize()
        if d_type not in ("Earthquake", "Typhoon", "Flood"):
            print("Please choose Earthquake, Typhoon, or Flood.")
            continue
        loc = input("Where is this happening? (e.g., Luzon, Visayas, Mindanao): ").strip()
        if not loc:
            print("Please enter a location.")
            continue
           hazard = input("Is the area near any specific hazards? (e.g., Coastline, River,
Mountain, None): ").strip() or "None"

        display_scale_guide(d_type)
        try:
            intensity = float(input(f"\nEnter the {d_type} Intensity Level (Number): "))
        except ValueError:
            print("Oops! That doesn't look like a valid number. Let's try again.")
            continue
        if not math.isfinite(intensity) or intensity <= 0:
            print("Please enter a finite, positive intensity.")
            continue
        if d_type in ("Typhoon", "Flood") and not 1 <= intensity <= 5:
            print("For this demonstration, Typhoon and Flood levels must be between 1 and 5.")
            continue

        category = get_intensity_category(d_type, intensity)


        print(f"\n[✔] NEW PROBLEM DESCRIPTION:")
         print(f"    Disaster: {d_type} | Location: {loc} | Intensity: {intensity} ({category})
| Hazard: {hazard}")

        new_event = {
            "disaster_type": d_type,
            "intensity": intensity,
            "location": loc,
            "category": category,
            "proximity_hazard": hazard
        }

        search_pool = copy.deepcopy(local_database)
        if d_type == "Earthquake":
            search_pool.extend(search_internet_for_cases(intensity))

        # --- PHASE 2: RETRIEVED SIMILAR CASE ---
        print("\n--- 2. ANALYZING PAST EXPERIENCES ---")
```

```
        similar_case, score = retrieve_similar_case(new_event, search_pool)
        if similar_case is None:
              print(f"No {d_type} cases are available. A different disaster type will not be
reused.")
            if input("Try another scenario? (y/n): ").strip().lower() == 'y':
                continue
            break

        # RUBRIC REQUIREMENT: "Retrieved similar case" & "Similarity score or ranking"
        print(f"✔ Most Similar Past Case Found: {similar_case['id']}")
        print(f"✔ Similarity Score: {score:.2f}")

         history = similar_case.get("historical_context", "We don't have a detailed historical
story for this specific case yet, but we will use its data!")
        print(f"\n📖 What happened back then:\n   {history}")

        # --- PHASE 3: ADAPTED FINAL SOLUTION ---
        print("\n--- 3. RECOMMENDED RESPONSE & COMMUNITY PLAN ---")
                    final_solution   =   adapt_solution(similar_case["solution"],   new_event,
similar_case["problem"])

        #"Adapted final solution" (evacuation, capacity, resources)
        print(f"\n🔴 PROPOSED EVACUATION STRATEGY (ADAPTED):")
        print(f"   - {final_solution['evacuation_strategy']}")

        print(f"\n🏠 SHELTER CAPACITY & FACILITIES (ADAPTED):")
        calculation = final_solution["capacity_adaptation"]
        print(f"   Original capacity: {calculation['original_capacity']:,} beds")
        print(f"   Recommended capacity: {final_solution['shelter_capacity']:,} beds")
        print(f"   Capacity increase: {calculation['increase_percent']}% (project rule)")
        print(f"   Facilities: {final_solution['shelter_description']}")

        print(f"\n📦 EMERGENCY RESOURCES TO DEPLOY (ADAPTED):")
        if isinstance(final_solution['resource_allocation'], dict):
            for category_name, items in final_solution['resource_allocation'].items():
                print(f"   [{category_name.upper()}]")
                for item in items:
                    print(f"      • {item}")

        print(f"\n💡 FRIENDLY TIPS & GUIDE FOR THE COMMUNITY:")
        for tip in final_solution.get("friendly_tips", []):
            print(f"   {tip}")

        final_solution = review_solution(final_solution)

        # --- PHASE 5: RETAIN THE USER-REVIEWED SOLUTION ---
        print("\n--- 5. SYSTEM LEARNING PHASE ---")
         if input("Would you like me to save this reviewed plan to my permanent memory? (y/n):
").strip().lower() == 'y':
            new_id = f"Local Memory - {d_type} in {loc} ({intensity})"
               new_history = f"Classroom scenario: {category.lower()} {d_type} in {loc}, near
{hazard}. The user reviewed the proposed shelter capacity; actual disaster outcomes have not
been recorded."

            local_database.append({
                "id": new_id,
                "historical_context": new_history,
                "problem": new_event,
                "solution": final_solution
            })
            save_database(local_database)
```

```
            print(f"\n[✔] Awesome! Successfully saved '{new_id}' (New Case Saved).")
                                        print(f"[✔]   Saved   reviewed   shelter   capacity:
{final_solution['shelter_capacity']:,} beds")


            print("\n[✔] UPDATED CASE LIST IN DATABASE:")
            for case in local_database:
                print(f"   - {case['id']}")

                 if  input("\nDo  you  need  to  analyze  another  disaster  scenario?  (y/n):
").strip().lower() != 'y':
            print("\nStay safe out there! Shutting down system. 👋")
            break

if __name__ == "__main__":
    main()
```